Experiment-5 OOPs in System Verilog-2

AIM:
To Understand and familiarize with OOPs in System Verilog including:
• Inheritance
• Polymorphism

SOFTWARE/TOOLS USED:
Synopsys VCS

Questions:

PART-A: Understanding Inheritance

You are required to model a Generic Electronic Device and a Smart Device that extends it.

1. Base Class – GenericDevice

Create a class named GenericDevice with the following:

- Integer property power_rating
- A integer variable device_count to keep track of the total number of devices created.
- A constructor that:
  - o Initializes power_rating with any value
  - o Increments device_count whenever a new object is created
- Function display() to print
  - o power_rating
  - o Total device_count

2. Derived Class – SmartDevice

Create a subclass **SmartDevice** that extends GenericDevice and includes:

- Integer property memory_size
- Constructor that:
  - o Initializes memory_size
  - o Calls the base class constructor

- Function display_memory() to print memory_size

3. Demonstration

Create three objects of SmartDevice

Assign different values for:

- power_rating
- memory_size

Call display functions from each object

Show that:

- Each object has its own power_rating and memory_size
- All objects share the same device_count

Modify device_count through one object and display from another object

PART-B: Accessing Base Class Data

You are designing a Memory System where a base class provides default memory size and a derived class modifies it

4. Base Class – MemoryBase

Create a class named MemoryBase with:

- Integer variable mem_size
- Constructor initializing mem_size=1024

Function display() to print memory size

5. Derived Class – MemoryController

Create a class MemoryController extending MemoryBase with

- A constructor that:
  - o Defines its own mem_size and initializes it with a value passed from the constructor
  - o Displays the derived class memory size
  - o Modifies the base class memory size with double the value of mem_size
  - o Displays the updated base class memory size

8. Demonstration

In the testbench::

1. Create an object of MemoryController with memory size = 2048 KB
2. Call the display() function
3. Observe and explain how base and derived values differ

PART-C: Polymorphism

You are modelling a Peripheral Device System where different peripherals respond differently to the same operation

9. Base Class – Peripheral
Create a class named **Peripheral** with:
- Virtual function operate()
- The function prints "Generic Peripheral Operation"

10. Derived Classes
Create the following derived classes:
1. **UART**
   - Override operate()
   - Print: "UART Transmitting Data"
2. **SPI**
   - Override operate()
   - Print: "SPI Communicating"
3. **I2C**
   - Override operate()
   - Print: "I2C Data Transfer"

11. Demonstration of Polymorphism
- Declare a base class handle of type Peripheral
- Create objects of UART, SPI, and I2C
- Assign each object to the base class handle one by one
- Call the operate() method through the base class handle
- Observe that the correct derived class method executes

THEORY:
SystemVerilog supports advanced Object-Oriented Programming (OOP) concepts that improve modularity, reusability, and scalability in verification environments. In this experiment, the concepts of **Inheritance, Accessing Base Class Data, and Polymorphism** are studied.

1. Inheritance

Inheritance allows a class (derived class) to acquire properties and methods of another class (base class) using the extends keyword. Inheritance promotes code reuse by establishing hierarchical relationships between classes, enabling extension of base functionality while reducing code redundancy.
In SystemVerilog, inheritance is implemented using:
class DerivedClass extends BaseClass;
The derived class inherits the data members, member functions, and static variables of the base class. The **super** keyword is used to call the base class constructor and to access base class variables and methods.
Static variables declared in a class are shared among all objects of that class. Hence, changes made through one object reflect in all other objects.

2. Accessing Base Class Data (Shadowing Concept)

When a derived class declares a variable with the same name as a base class variable, it results in **variable shadowing**.
Example: Base class: mem_size, Derived class: mem_size
In such cases, *this.mem_size* refers to derived class variable and *super.mem_size* refers to base class variable.

This allows independent modification of base and derived data, controlled access to inherited members, and a clear separation between base and extended behavior.
The super keyword ensures that the base class data can still be accessed even when shadowed.

3. Polymorphism

Polymorphism allows a base class handle to refer to objects of derived classes.
In SystemVerilog, polymorphism is implemented using: ***virtual*** *function*

A virtual function enables **dynamic binding**, meaning the function call is resolved at runtime based on the actual object assigned to the base class handle.

A base class handle can point to derived class objects, and the method executed depends on the actual object assigned at runtime. This enables flexible and scalable verification models and is essential for advanced verification methodologies like UVM. This mechanism is known as runtime polymorphism.

System Verilog Code with Comments (#):

PART A:

```
//INHERITANCE DEMONSTRATION

//==================================

==========================

// Base Class – GenericDevice

//-------------------------------------------------------

--

class GenericDevice;


  int power_rating;

  static int device_count = 0;


  function new(int power_rating);

    this.power_rating = power_rating;

    device_count++;

  endfunction

  function void display();

    $display("Power Rating        : %0d",

power_rating);

    $display("Total Device Count  : %0d",

device_count);

  endfunction


endclass

yyyy

//-------------------------------------------------------

--

// Derived Class – SmartDevice

//-------------------------------------------------------

--
```

```
class SmartDevice extends GenericDevice;
  int memory_size;

  function new(int power_rating, int
memory_size);
    super.new(power_rating);
    this.memory_size = memory_size;
  endfunction

  function void display_memory();
    $display("Memory Size        : %0d",
memory_size);
  endfunction

  function void display_full();
    $display("----------------------------------------");

    display();
    display_memory();
    $display("----------------------------------------");
  endfunction

endclass
//-------------------------------------------------------------
--
```

```
// Testbench – Part A
//-------------------------------------------------------------
--
module partA_tb;
  initial begin
    SmartDevice sm1 = new(50, 128);
    SmartDevice sm2 = new(75, 256);
    SmartDevice sm3 = new(100, 512);

    $display("\nPART – A OUTPUT\n");

    sm1.display_full();
    sm2.display_full();
    sm3.display_full();

    sm1.device_count = 10;

    $display("\nAfter modifying device_count via
sm1:\n");
    sm2.display_full();

  end

endmodule
```

PART B:

```
//=================================================================
//                    PART – B
//            ACCESSING BASE CLASS DATA
//=================================================================

//-----------------------------------------------------------------
// Base Class – MemoryBase
//-----------------------------------------------------------------
class MemoryBase;

  int mem_size;

  function new();
    this.mem_size = 1024;
  endfunction

  function void display();
    $display("Memory size in base class MemoryBase : %0d KB", mem_size);
  endfunction
endclass

//-----------------------------------------------------------------
// Derived Class – MemoryController
//-----------------------------------------------------------------
class MemoryController extends MemoryBase;

  int mem_size;

  function new(int size);

    super.new();
    this.mem_size = size;

    $display("\nDerived Memory Size          : %0d KB", this.mem_size);

    super.mem_size = 2 * this.mem_size;

    $display("Base Memory Size (after doubling) : %0d KB", super.mem_size);
  endfunction
```

```
endclass

//-------------------------------------------------------------
--
// Testbench – Part B
//-------------------------------------------------------------
--
module partB_tb;

  initial begin
    $display("\nPART – B OUTPUT\n");

    MemoryController mc = new(2048);

    $display("\nCalling Base Class display():");
    mc.display();

  end

endmodule
```

PART C:

```
//=================================
==========================
//                PART – C
//              POLYMORPHISM
//=================================
==========================

//-------------------------------------------------------------
--
// Base Class – Peripheral
//-------------------------------------------------------------
--
class Peripheral;
  virtual function void operate();
    $display("Generic Peripheral Operation");
  endfunction
endclass

//-------------------------------------------------------------
--
// Derived Classes
//-------------------------------------------------------------
--
```

```
class UART extends Peripheral;
  function void operate();
    $display("UART Transmitting Data");
  endfunction
endclass

class SPI extends Peripheral;
  function void operate();
    $display("SPI Communicating");
  endfunction
endclass

class I2C extends Peripheral;
  function void operate();
    $display("I2C Data Transfer");
  endfunction
endclass

//-------------------------------------------------------------
--
// Testbench – Part C
//-------------------------------------------------------------
--
module partC_tb;

  Peripheral p;

  initial begin

    $display("\nPART – C OUTPUT\n");

    UART u = new();
    SPI  s = new();
    I2C  i = new();

    p = u; p.operate();
    p = s; p.operate();
    p = i; p.operate();

  end

endmodule
```

RESULTS:

PART A:

```
Compiler version V-2023.12-1_Full64; Runtime version V-2023.12-1_Full64;  Feb 20 23:47 2026

================================================
                PART – A OUTPUT
        INHERITANCE DEMONSTRATION
================================================

Device 1 Details:
----------------------------------------
Power Rating        : 50
Total Device Count  : 3
Memory Size         : 128
----------------------------------------
Device 2 Details:
----------------------------------------
Power Rating        : 75
Total Device Count  : 3
Memory Size         : 256
----------------------------------------
Device 3 Details:
----------------------------------------
Power Rating        : 100
Total Device Count  : 3
Memory Size         : 512
----------------------------------------

Modifying device_count via Device 1...

Verifying through Device 2:
----------------------------------------
Power Rating        : 75
Total Device Count  : 10
Memory Size         : 256
----------------------------------------
Observation:
• Each object has independent power_rating and memory_size.
• device_count is shared among all objects.
```

Inheritance is demonstrated, showing that instance variables are independent while the static variable device_count is shared among all objects.

PART B:

```
================================================
                PART – B OUTPUT
        BASE CLASS DATA ACCESS
================================================

Derived Class Initialization:
----------------------------------------
Derived Memory Size          : 2048 KB
Base Memory Size (after doubling) : 4096 KB
Derived Memory Size (unchanged)   : 2048 KB
----------------------------------------

Calling display() from Base Class:

Memory size in base class MemoryBase : 4096 KB

Observation:
• Derived class has its own mem_size.
• Base class mem_size modified using super.
• Shadowing behavior demonstrated.
```

Base class data access and variable shadowing were validated using the super keyword.

PART C:

```
================================================
                PART – C OUTPUT
                 POLYMORPHISM
================================================

Base Handle → UART Object:
----------------------------------------
UART Transmitting Data
----------------------------------------

Base Handle → SPI Object:
----------------------------------------
SPI Communicating
----------------------------------------

Base Handle → I2C Object:
----------------------------------------
I2C Data Transfer
----------------------------------------

Observation:
• operate() executes according to actual object assigned.
• Demonstrates runtime polymorphism using virtual functions.
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.110 seconds;       Data structure size:   0.0Mb
Fri Feb 20 23:47:10 2026
CPU time: .102 seconds to compile + .086 seconds to elab + .106 seconds to link + .123 seconds in simulation
[c2user34@vlsi34 2026-02-17_eval5]$
```

Runtime polymorphism was confirmed as the operate() method executed according to the actual object assigned to the base class handle.

INFERENCE:

In this experiment, the advanced OOP concepts of Inheritance and Polymorphism in SystemVerilog were successfully implemented and verified. It was observed that derived classes inherit properties and methods from the base class, while static variables are shared among all objects. The use of the super keyword enabled access to base class data even in the presence of variable shadowing. Runtime polymorphism was demonstrated using virtual functions, where the method execution depended on the actual object assigned to the base class handle. Thus, the experiment confirms that OOP concepts enhance modularity, reusability, and flexibility in hardware verification environments.